Lab 3 Report

MPI Collective Operations

Name: Ahmad Bin Rashid

Roll No: 2023-CS-53

Instructor: Sir Waqas Ali

Course: Distributed and Parallel Computing

University of Engineering and Technology

March 12, 2026

1 Exercise Execution Screenshots

1.1 Exercise 1: MPI Broadcast

```
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 4 ./bcast_example
Process 0 broadcasting value 1000
Process 1 received data = 1000
Process 2 received data = 1000
Process 3 received data = 1000
Process 0 received data = 1000
```

Figure 1: Successful execution of the MPI Broadcast example

1.2 Exercise 2: MPI Reduce

```
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 1 ./reduce_sum
The sum from 1 to 1000 is: 500500
Time with 1 processes: 0.000005 seconds
Expected sum: 500500
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 2 ./reduce_sum
The sum from 1 to 1000 is: 500500
Time with 2 processes: 0.000011 seconds
Expected sum: 500500
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 4 ./reduce_sum
The sum from 1 to 1000 is: 500500
Time with 4 processes: 0.000029 seconds
Expected sum: 500500
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 8 --oversubscribe ./reduce_sum
The sum from 1 to 1000 is: 500500
Time with 8 processes: 0.000046 seconds
Expected sum: 500500
```

Figure 2: Execution output for MPI Reduce example

1.3 Exercise 3: MPI Scatter and Gather

```
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 4 ./scatter_gather
Process 2: local sum = 156375
Process 3: local sum = 218875
Process 0: local sum = 31375
Process 1: local sum = 93875
Total sum via MPI_Reduce: 500500
Collected sums: 31375 93875 156375 218875
Expected total sum = 500500
```

Figure 3: Execution output for Scatter and Gather example

1.4 Exercise 4: MPI Scatterv and Gatherv

```
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 3 ./scatterv_gatherv

Process 1: received 333 elements, local sum = 166833
Process 2: received 333 elements, local sum = 277722
Process 0: received 334 elements, local sum = 55945
Total sum via MPI_Reduce = 500500
Expected total sum = 500500
MPI_Gatherv verification: PASS
```

Figure 4: Execution output demonstrating uneven distribution using Scatterv/Gatherv

1.5 Exercise 5: Matrix Vector Multiplication

```
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 4 ./matvec
Result vector y:
8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000
ahmad@Pakiman92:~/study/pdc lab/lab3$
ahmad@Pakiman92:~/study/pdc lab/lab3$ mpirun -np 4 ./matvec_v
Result vector y (M=10, N=8, 4 processes):
8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000
```

Figure 5: Matrix-vector multiplication result using MPI

2 Code Listing for Exercise 4 Modifications

The following code modifications were made to correctly implement `MPI_Scatterv` and `MPI_Gatherv` for uneven data distribution.

```c
// Key modifications for Scatterv/Gatherv implementation

// Fix broken declarations
int *sendcounts;
int *displs;

// Compute sendcounts and displacements for all processes
for (int i = 0; i < size; i++) {
    sendcounts[i] = N / size;
    if (i < N % size)
        sendcounts[i]++;

    displs[i] = sum;
    sum += sendcounts[i];
}

// Scatter uneven chunks
MPI_Scatterv(data, sendcounts, displs, MPI_INT,
             local_data, local_n, MPI_INT,
```

```
              0, MPI_COMM_WORLD);

// Gather results
MPI_Gatherv(local_data, local_n, MPI_INT,
            gathered_data, sendcounts, displs,
            MPI_INT, 0, MPI_COMM_WORLD);
```

Explanation of Fixes and Additions:

- Fixed broken variable declarations caused by comment formatting errors.
- Corrected the loop responsible for calculating `sendcounts` and `displs`.
- Ensured that all processes compute their own `local_n` to avoid an extra `MPI_Scatter`.
- Added `MPI_Gatherv` to collect uneven chunks back to the root process.
- Implemented verification to confirm gathered data matches the original dataset.

3 Performance Comparison for MPI Reduce

Processes	Execution Time (seconds)	Result
1	0.000005	500500
2	0.000011	500500
4	0.000029	500500
8	0.000046	500500

Table 1: Execution time comparison for MPI Reduce example

4 Discussion: Collective vs Point-to-Point Communication

Collective operations in MPI provide a higher-level abstraction for common communication patterns such as broadcasting, reduction, and data distribution. Unlike point-to-point communication, where the programmer must explicitly manage message exchanges between processes, collective operations allow multiple processes to participate in a coordinated communication step.

These operations are usually optimized internally by the MPI implementation, resulting in better scalability and performance. They also simplify program development by reducing the complexity of synchronization and message management. In contrast, point-to-point communication offers more flexibility but requires careful coordination to avoid issues such as deadlocks and inefficient communication patterns.

5 Self-Assessment Answers

1. What is the main difference between point-to-point and collective communication?
Point-to-point communication involves explicit message exchange between two specific processes using functions like `MPI_Send` and `MPI_Recv`. Collective communication involves a group of processes participating in a coordinated operation such as `MPI_Bcast` or `MPI_Reduce`.

2. Why must all processes call a collective operation like MPI_Bcast?
All processes in the communicator must call the collective function because MPI synchronizes them internally. If some processes skip the call, the program may hang or produce undefined behavior.

3. In the matrix-vector multiplication example, why do we broadcast `x` instead of scattering it?
The vector `x` is needed entirely by every process to compute its portion of the result vector. Broadcasting ensures all processes receive the full vector.

4. Write the MPI_Reduce call that computes the maximum value across all processes.

```
MPI_Reduce(&local_value, &max_value, 1, MPI_INT, MPI_MAX, 0,
    MPI_COMM_WORLD);
```

5. Which collective would you use to collect processed results back to process 0?
`MPI_Gather` would be used to collect the processed data from all processes and store it in a receive buffer on process 0.

6. What happens if the root in MPI_Gather provides a receive buffer that is too small?
The program may cause memory corruption, incorrect results, or a runtime error because incoming data exceeds the allocated buffer.

7. In the scatterv example, explain the purpose of `sendcounts` and `displs`.
`sendcounts` specifies how many elements each process receives, while `displs` specifies the starting position of each chunk in the original array.

8. Why is MPI_Reduce usually faster than manual MPI_Send/MPI_Recv implementations?
MPI collective operations use optimized algorithms (such as tree-based reductions) that minimize communication steps, making them more efficient than manual implementations.

9. How would you modify the matrix-vector multiplication to use MPI_Allgather?
Each process would compute its partial result and use `MPI_Allgather` instead of `MPI_Gather` so that every process receives the complete result vector `y`.